PROYECTO 2

Apellidos, Nombre: Flores Flores, Daniel Javier
Apellidos, Nombre: Lillo Ramirez, Lucas
Apellidos, Nombre:

La entrega definitiva del proyecto será el 9 de marzo. En la entrega subid el fichero lab.m con vuestro código junto con este fichero (doc/pdf) con las respuestas/código e imágenes pedidas.

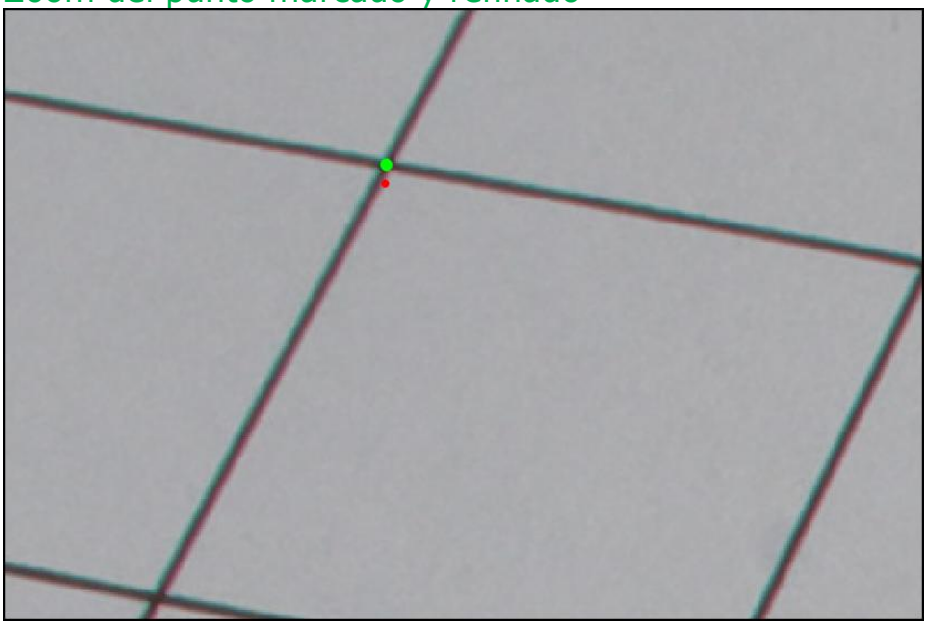
Para la clase del 2 de marzo debéis llevar hecho hasta el punto indicado. En esa clase revisaré/puntuaré lo que tengáis hecho, resolveremos dudas, y seguiréis trabajando con el resto de los apartados, ...

1) Determinación de la matriz H

Adjuntad volcado con las posiciones iniciales y finales de las esquinas, así como un zoom de una de las esquinas donde se aprecie el punto inicial marcado (rojo) y su posición refinada (verde).

Pincha esquina superior izda :x=1309.4,y= 660.1
Coordenadas refinadas: x=1310.0,y= 651.4
Pincha esquina superior drcha:x=4084.1,y=1186.6
Coordenadas refinadas: x=4077.2,y=1190.9
Pincha esquina inferior drcha:x=4040.8,y=3001.3
Coordenadas refinadas: x=4031.6,y=2999.5
Pincha esquina inferior izda :x= 547.6,y=2146.6
Coordenadas refinadas: x= 538.4,y=2151.3

Zoom del punto marcado y refinado



Adjuntad el código de la función refinar().

```
function [x,y]=refinar(x,y,aux)
R=50; % Definici►n del tama►o de la zona a explorar
rr=(-R:R); cx=ones(length(rr),1)*rr; cy=cx';

  % redondeamos las coordenadas iniciales X, Y
  x=round(x);
  y=round(y);
  % extraemos subimagen s
  s = aux(y-R:y+R,x-R:x+R);
  % Convertimos s a double y calculamos su valor minimo
  s = double(s);
  m = min(s(:));
  % Diferencias respecto al valor minimo
  d = abs(s - m);
  % Calculamos pesos y obtenemos matriz de pesos
  w = exp(-d);
  w = w/sum(w(:));
  % Calculamos el desplazamiento
  x = x + sum(sum(cx.*w));
  y = y + sum(sum(cy.*w));

end
```

Adjuntad volcado de H.

Matriz H:

13.998	8.759	2427.41
2.252	-10.319	1612.12
-0.001	0.002	1.00

Adjuntad los valores de X,Y usados junto con la nueva matriz H obtenida.

Los valores X, Y usados son :

```
X1 = [80,100,100,80];
Y1 = [60,60,40,40];
```

Matriz H2:

15.111	8.786	2461.86
3.020	-10.367	1634.42
-0.000	0.002	1.00

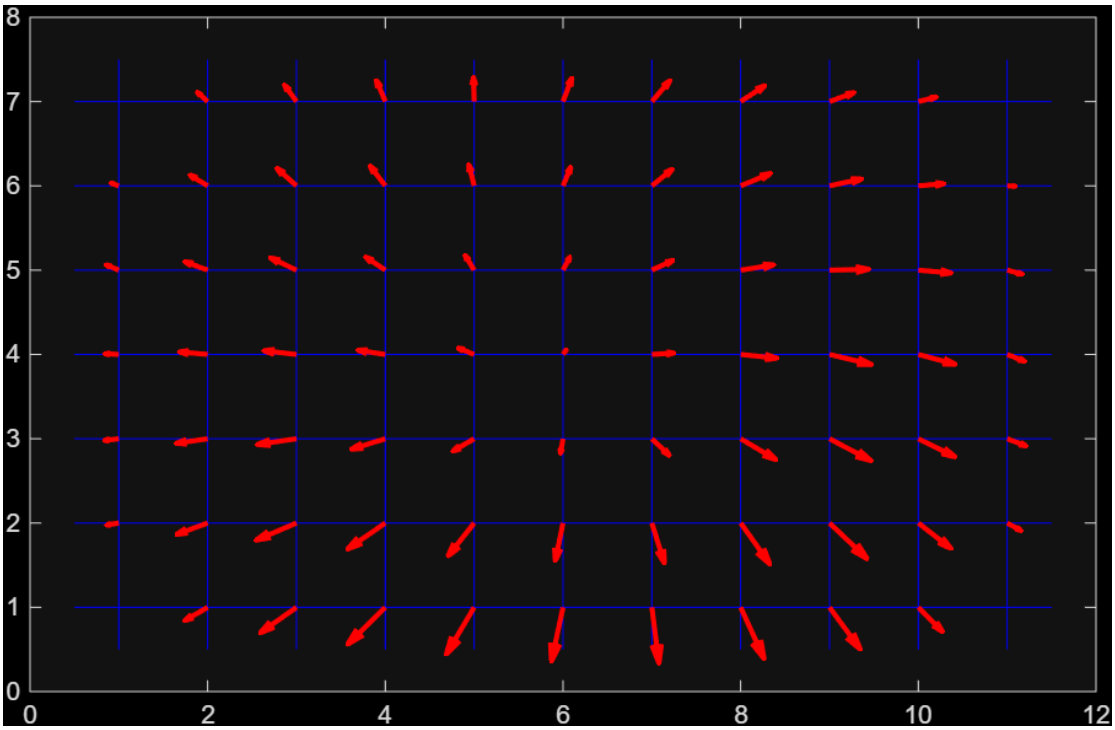
¿Cuál de las dos estimaciones de H creéis que será más adecuada para usar en la calibración de la cámara? Justificad.

La primera es más adecuada ya que un error pequeño no destaca tanto pero en la segunda que un error proporcionalmente sería mayor, por lo que destacaría más y generaría más problemas si lo intentas usar como referencias.

¿Cuál es la distancia media entre los puntos "predichos" y los "correctos"? Adjuntad un plot del vector de distancias d.

Distancia media entre predichos y refinados: 10.644 píxeles

Adjuntad el gráfico obtenido. ¿Qué puntos tienen el menor error en la gráfica? ¿A qué se debe esto?



Volcad la nueva matriz H usando los 77 puntos.
Nueva matriz H (77 puntos):

14.492	8.655	2448.81
2.664	-10.309	1634.26
-0.000	0.002	1.00

Llevar hecho hasta aquí para la clase del 2 de marzo.

2) Estimación de la focal f y pose (posición/giro) de la cámara

Indicad los valores u0, v0 a usar y volcad la focal f obtenida.
A continuación se muestran los valores mostrados el ejecutar nuestro código
Usamos u0=2464, v0=1632.0 (px)
La f local que obtenemos es: f = 3500.87 px

Volcar la matriz Q obtenida.

Matriz Q:

0.004	0.001	-0.00
0.001	-0.004	0.00
-0.000	0.002	1.00

Volcad los valores obtenidos para n_1,n_2 y λ y los vectores corregidos r1, r2 y t.

Los valores obtenidos son:
n1 = 0.004556
n2 = 0.004521
lambda = 0.004538
r1 corregido:
0.9772
0.2107
-0.0924

r2 corregido:
0.2278
-0.8588
0.4504

t corregido:
-0.9561
0.1422
220.3443

Usad los tres vectores columnas para construir R y volcad el resultado.

El valor obtenido es:
Matriz R :

0.9772	0.2278	0.0155
0.2107	-0.8588	-0.4611
-0.0924	0.4504	-0.8872

¿De qué orden son las diferencias entre los productos que habéis obtenido y la matriz identidad?
Los productos obtenidos son los siguientes

R*R' =

1.0070	0.0031	-0.0015
0.0031	0.9946	0.0029
-0.0015	0.0029	0.9985

R'*R =

1.0078	0.0000	0
0.0000	0.9923	0
0	0	1.0000

Por lo que cuando calculamos las diferencias se nos queda

```
||R'R - I|| =
    0.0070    0.0031   -0.0015
    0.0031   -0.0054    0.0029
   -0.0015    0.0029   -0.0015

||RR' - I|| =
    0.0078    0.0000         0
    0.0000   -0.0077         0
         0         0    0.0000
```

Las diferencias entre los productos de R y R traspuesta (y viceversa) que hemos obtenido están entre 10^-3 y 10^-2. Si tenemos en cuent que las desviaciones más grandes se sitúan entorno a 10^-2, podemos decir que **el orden de las diferencias es de 10^-2**.

Adjuntad el vector X_0 obtenido. En el momento de tomar la foto, ¿a qué distancia se encontraba el "centro óptico" de la cámara del punto central de la malla en el papel?

A continuación, se adjunta el valor del vector X0 que obtenemos

```
X0                                                                    =

   21.2664
  -98.8960
  195.5694
```

Para saber a qué distancia se encontraba el centro óptico respecto del papel tenemos que calcular la norma del vector X0 en este caso;

```
>> norm(X0)

ans =

  220.1818
```

Con esto podemos decir que el centro óptico se encontraba a una distancia de unos 220.2 mm del punto central de la malla. Hemos puesto mm porque es lo más coherente teniendo en cuenta la foto y también que previamente se decía que las medidas de los cuadrados de la malla estaban en mm.

Adjuntad código de vuestra función.

```
function [f,R,X0]=get_data_from_H(H)
  % Definir valores de u0, v0
  im=imread('malla.jpg');
  [N,M,~]=size(im);
  u0 = M/2;
  v0 = N/2;
 fprintf('Usamos u0=%1.f, v0=%.1f   (px)\n', u0, v0);
  % Construimos la matriz B
  B = [1 0 -u0; 0 1 -v0; -u0 -v0 (u0^2 + v0^2)];
  % Obtenemos las dos primeras columnas de la matriz H y (H31 H32 H33)
  h1 = H(:,1);
  h2 = H(:,2);
  H31 = H(3,1); H32 = H(3,2); H33 = H(3,3);
  % Calculamos f
  numerador = -(h1.' * B * h2);
  denominador = (H31 * H32 * H33);
  f = sqrt(numerador / denominador);
  fprintf('La f local que obtenemos es: f = %.2f px\n', f);

  % Calculamos K
  K = [f 0 u0;0 f v0; 0 0 1];
  Q = K \ H; % Calculamos Q = K^-1 * H
  fprintf('Matriz Q:\n');
  vuelca_matriz(Q);
  fprintf('\n');

  % Extraemos r1, r2 y t
  r1 = Q(:,1);
  r2 = Q(:,2);
  t = Q(:,3);
  % Calculamos sus normas
  n1 = norm(r1);
  n2 = norm(r2);
  lambda = sqrt(n1*n2);
  % Corregir escala
  r1 = r1 / lambda;
  r2 = r2 / lambda;
  t  = t  / lambda;
  fprintf('n1 = %.6f\n', n1);
  fprintf('n2 = %.6f\n', n2);
  fprintf('lambda = %.6f\n', lambda);
  fprintf('r1 corregido:\n'); disp(r1);
  fprintf('r2 corregido:\n'); disp(r2);
  fprintf('t corregido:\n');  disp(t)

  % Tercera columna de R y construcción de R
  r3 = cross(r1, r2);
```

```
  R  = [r1 r2 r3];
  fprintf('Matriz R :\n');
  disp(R);

  % Comprobamos los resultados de R*R' y R'*R
  Ident = eye(3);
  M1 = R*R'; M2 = R'*R;
  fprintf('R*R'' =\n'); disp(M1);
  fprintf('R''*R =\n'); disp(M2);
  Dif1 = M1 - Ident;
  Dif2 = M2 - Ident;
  fprintf('||R''R - I|| = \n'); disp(Dif1);
  fprintf('||RR'' - I|| = \n'); disp(Dif2);

  % Despejamos X0 a partir de R y t
  X0 = -R' * t

end
```

3) Mejora de la estimación de f, R y X_0 usando optimización

Adjuntad la matriz de rotación R obtenida del vector w con coordenadas (2, 0.5, 1) y el código de la función convertir_RW() una vez verificada.

```
Matriz de rotacion R
    0.5472   -0.0292    0.8365
    0.6494   -0.6156   -0.4464
    0.5280    0.7875   -0.3179


function out=convertir_Rw(in)

Ndata=numel(in);

if Ndata==9   % Conversion R --> w

    R = in;

    theta = acos((trace(R)-1)/2);

    if abs(theta) < 1e-10
        w = [0;0;0];
    else
        wx = (R(3,2)-R(2,3))/(2*sin(theta));
        wy = (R(1,3)-R(3,1))/(2*sin(theta));
        wz = (R(2,1)-R(1,2))/(2*sin(theta));

        w = theta * [wx; wy; wz];
```

```
    end

    out = w;

else   % Conversion w --> R

    w = in;

    theta = norm(w);

    if theta < 1e-10
        R = eye(3);
    else

        k = w/theta;

        K = [  0    -k(3)  k(2);
             k(3)   0    -k(1);
            -k(2)  k(1)   0   ];

        R = eye(3) + sin(theta)*K + (1-cos(theta))*(K*K);

    end

    out = R;

end
end
```

Adjuntad código de la función error_uv().

```
function err=error_uv(P,X,Y,u,v)

% Reservar vectores
N = length(u);
up = zeros(1,N);
vp = zeros(1,N);

% Extraer parametros
w  = P(1:3);
X0 = P(4:6);
f  = P(7);

% Obtener matriz de rotacion
R = convertir_Rw(w);

% Vector traslacion
t = -R*X0;
```

```
% Matriz Q
Q = [R(:,1) R(:,2) t];

% Coordenadas homogeneas del plano
XY = [X; Y; ones(1,N)];

% Coordenadas en camara
XYZ_cam = Q * XY;

Xcam = XYZ_cam(1,:);
Ycam = XYZ_cam(2,:);
Zcam = XYZ_cam(3,:);

% Coordenadas normalizadas
xn = Xcam ./ Zcam;
yn = Ycam ./ Zcam;

% centro imagen
im = imread('malla.jpg');
[Nim,Mim,~] = size(im);

u0 = Mim/2;
v0 = Nim/2;

% pasar a pixeles
up = u0 + f * xn;
vp = v0 + f * yn;

% errores
du = u - up;
dv = v - vp;

% vector error
err = [du dv]';

end
```

¿Cuál es la norma del vector de errores obtenido?

Norma del vector de errores: 1678.470

Dad la focal f obtenida, así como los vectores w y X0 (2 decimales).

Focal optimizada: 3619.961 px

```
w_opt                                                    =

    2.63
    0.31
   -0.05
```

X0_opt =

21.73
-104.99
200.08

Re-calcular la norma del vector de errores (que ahora debería ser menor).
Norma del vector de errores post-optimizacion: 59.791

¿Cuál es la densidad (en píxeles por mm) del sensor? Dad con un decimal los resultados de la focal de la lente (en mm) antes y después de hacer la optimización.
Densidad de píxeles inicial: 207.932 px/mm

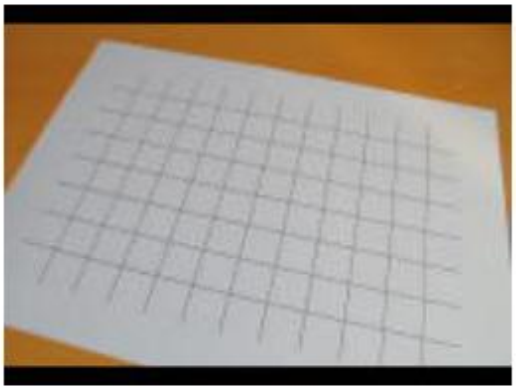
Focal inicial en mm: 16.837 mm

Focal optimizada en mm: 17.409 mm

¿Qué focal registró la cámara al tomar la foto? Adjuntad captura de pantalla con los datos mostrados.

DATOS EXIF DE LA IMAGEN

Fabricante de la cámara :	PENTAX
Modelo de la cámara :	PENTAX K-50
Fecha y hora :	09/02/2019 12:49:42
Resolución :	4928 x 3264
Usó flash :	No
Distancia Focal :	18.0mm (35mm equivalent 27mm)
Tiempo de exposición :	0.250 s (1/4)
Apertura :	f/16.0
ISO equiv. :	100
Exposure bias :	1.00
Balance de blancos :	Auto
Modo de medida :	pattern
Exposición :	aperture priority (semi-auto)
Exposure Mode :	Manual
Focus range :	macro
Latitud GPS :	? ?º ' "
Longitud GPS :	? ?º ' "
JPEG Quality :	94



Quitar Exif | Otra foto

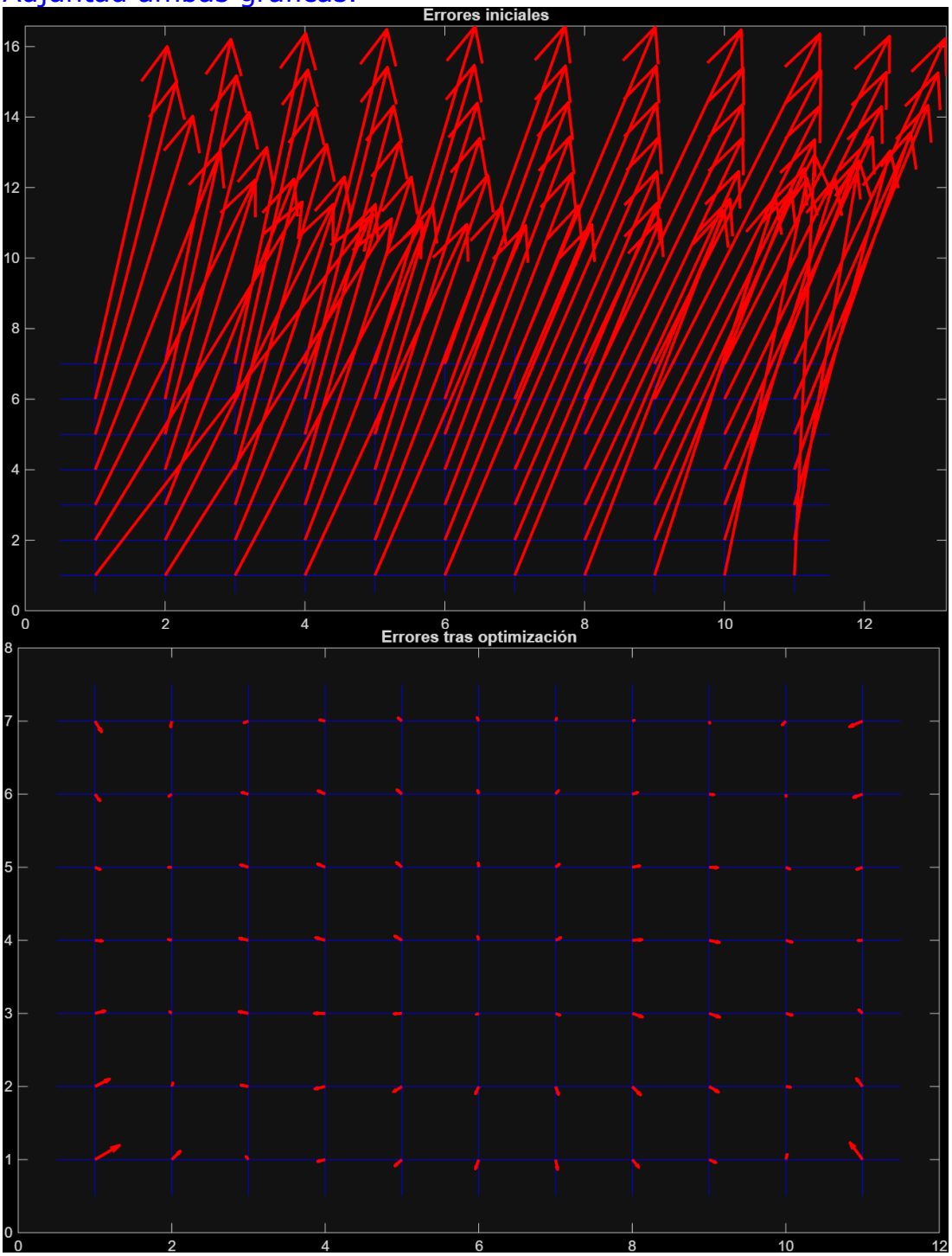
Dad los resultados de la estimación de la focal f, ω y X0 usando la nueva matriz H.
Focal inicial f: 3500.872 px
Vector de rotacion w:
2.5939
0.3072
-0.0486

Centro de la camara X0:
21.2664

-98.8960
195.5694

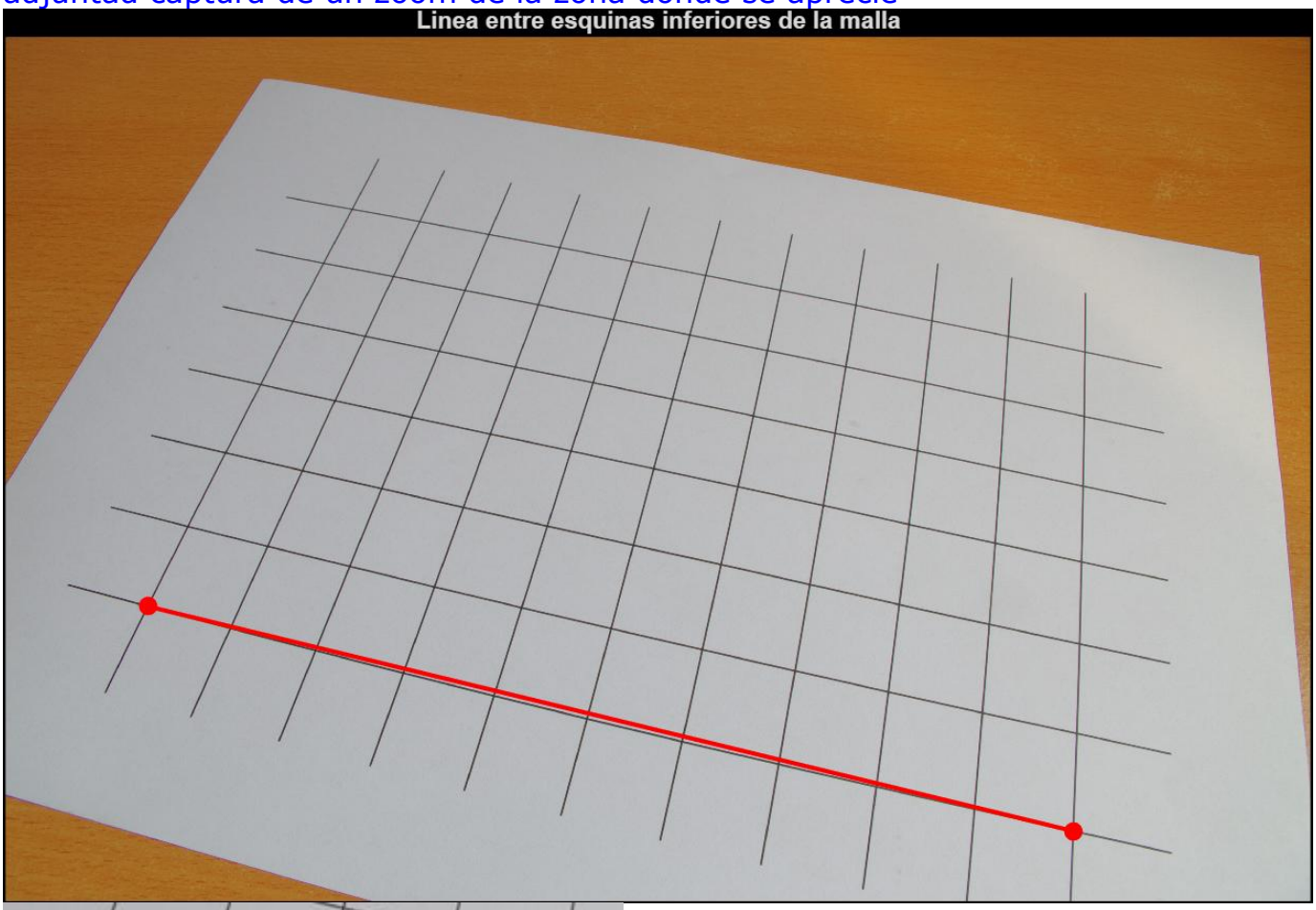
Evaluar como antes la función de errores error_uv() para esos parámetros y dad la norma del vector de errores inicial. Aplicad luego la optimización e indicad la norma del vector de errores a los que llegamos.
Norma del vector de errores inicial: 1678.470
Focal optimizada: 3619.961 px
Focal optimizada en mm: 17.409 mm

Adjuntad ambas gráficas.



Estimación de la distorsión

adjuntad captura de un zoom de la zona donde se aprecie



La segunda imagen es una captura de windows, no una funcion de matlab
Adjuntad el código modificado de la función error_uv.
Para no duplicar el código, adjuntamos únicamente los cambios:

```
% Extraer parametros
w  = P(1:3);
X0 = P(4:6);
f  = P(7);

if numel(P) >= 8
```

```
   k1 = P(8);
else
   k1 = 0;
end
```

Y luego también realizamos cambios en:

```
% Coordenadas normalizadas
xn = Xcam ./ Zcam;
yn = Ycam ./ Zcam;

r2 = xn.^2 + yn.^2;

xn = xn .* (1 + k1 * r2);
yn = yn .* (1 + k1 * r2);
```

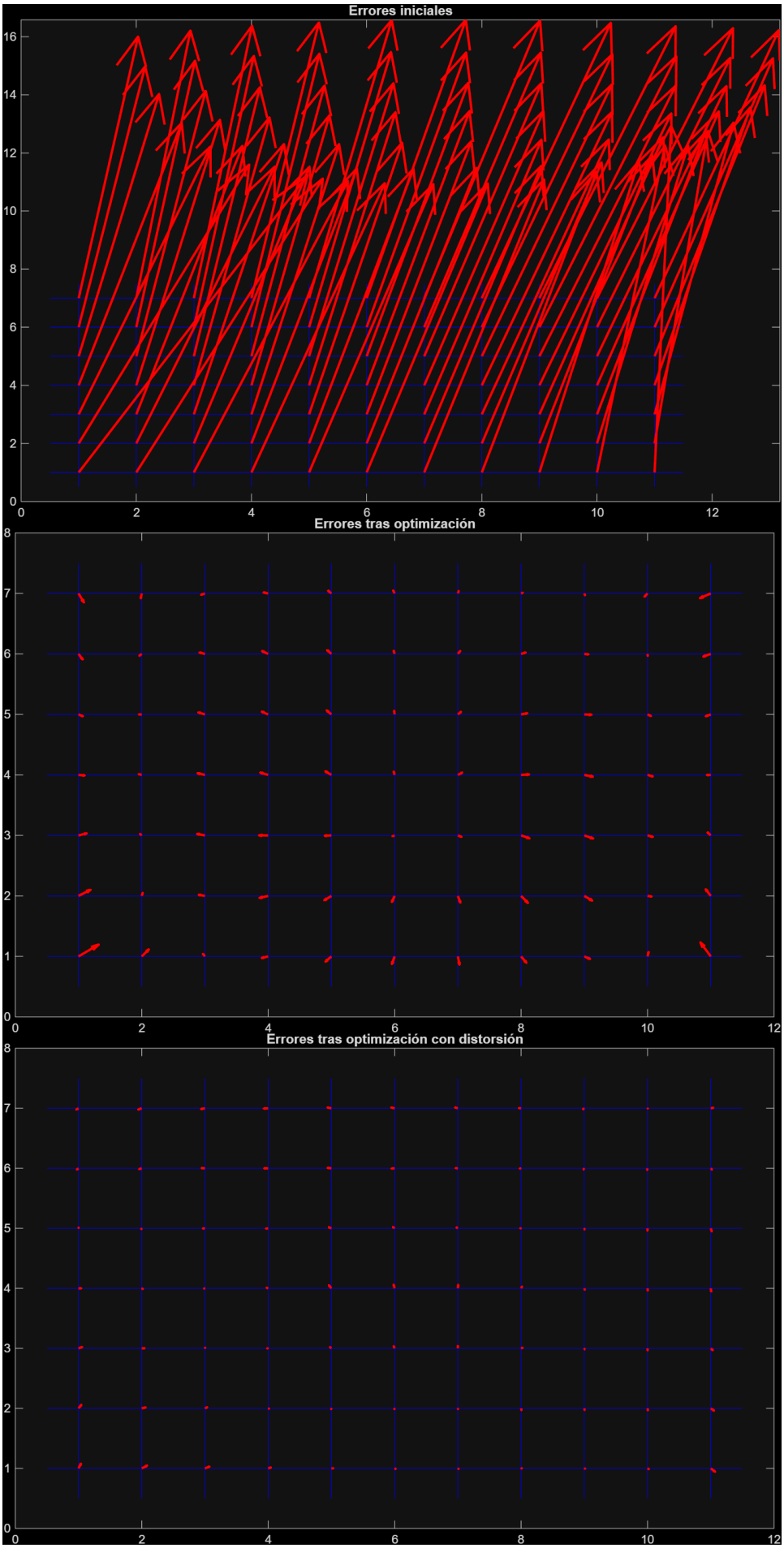
¿Valores de f y k1 obtenidos tras optimizar?

```
f_k1                                                    =

  3.5897e+03


k1                                                      =

  -0.0769
```

Rellenar la siguiente tabla con los valores de la norma del vector de errores para la hipótesis inicial y las dos optimizaciones (con y sin considerar distorsión).

	R, X_0 y f deducidos de H_{77}	Optimización	Optimización con k1
‖e‖ = norm(e)	1678.470	59.791	15.636

Usando show_err_malla() adjuntad 3 gráficas mostrando la evolución de los errores (errores de la hipótesis inicial, los errores tras la optimización inicial y los que quedan tras la optimización incorporando k1).


Errores iniciales
Errores tras optimización
Errores tras optimización con distorsión

4) Estimación de los parámetros de la cámara de vuestro teléfono

Marca/modelo de vuestro teléfono
Nothing Phone 2
Buscad información sobre su cámara:

- Número de píxeles del sensor

50MPX, aunque la foto se tomo con el sensor de 12MPX ya que teníamos dicha función activa.

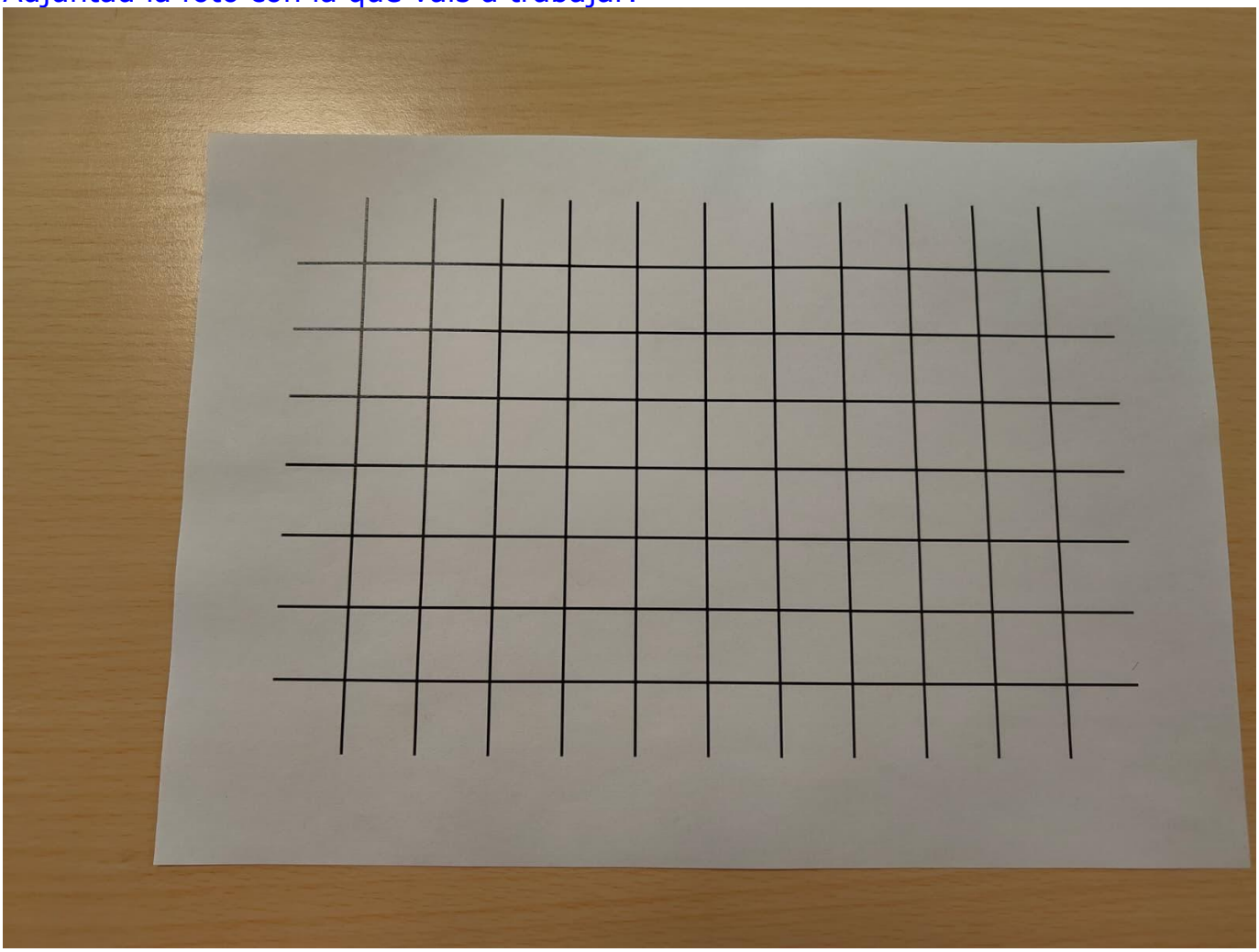
- Tamaño físico del sensor (ancho x alto en mm)

8.2 mm × 6.1 mm

- Focal de la lente (mm) de la cámara principal, ...

5.4 mm

Adjuntad la foto con la que vais a trabajar.



Indicad los parámetros que habéis tenido que cambiar en el programa

Hemos modificado valores como la imagen(que cambia luego el size de la imagen),el radio de refinamiento y el tamaño del sensor

Dad los resultados encontrados, como:

- H estimada usando las 4 esquinas y usando todos los puntos de la malla.

- Matriz H:

```
4.502   0.448   898.26
0.067  -4.097   588.10
0.000   0.001     1.00
```

- Nueva matriz H77:

```
4.506   0.449   898.97
0.065  -4.100   588.46
0.000   0.001     1.00
```

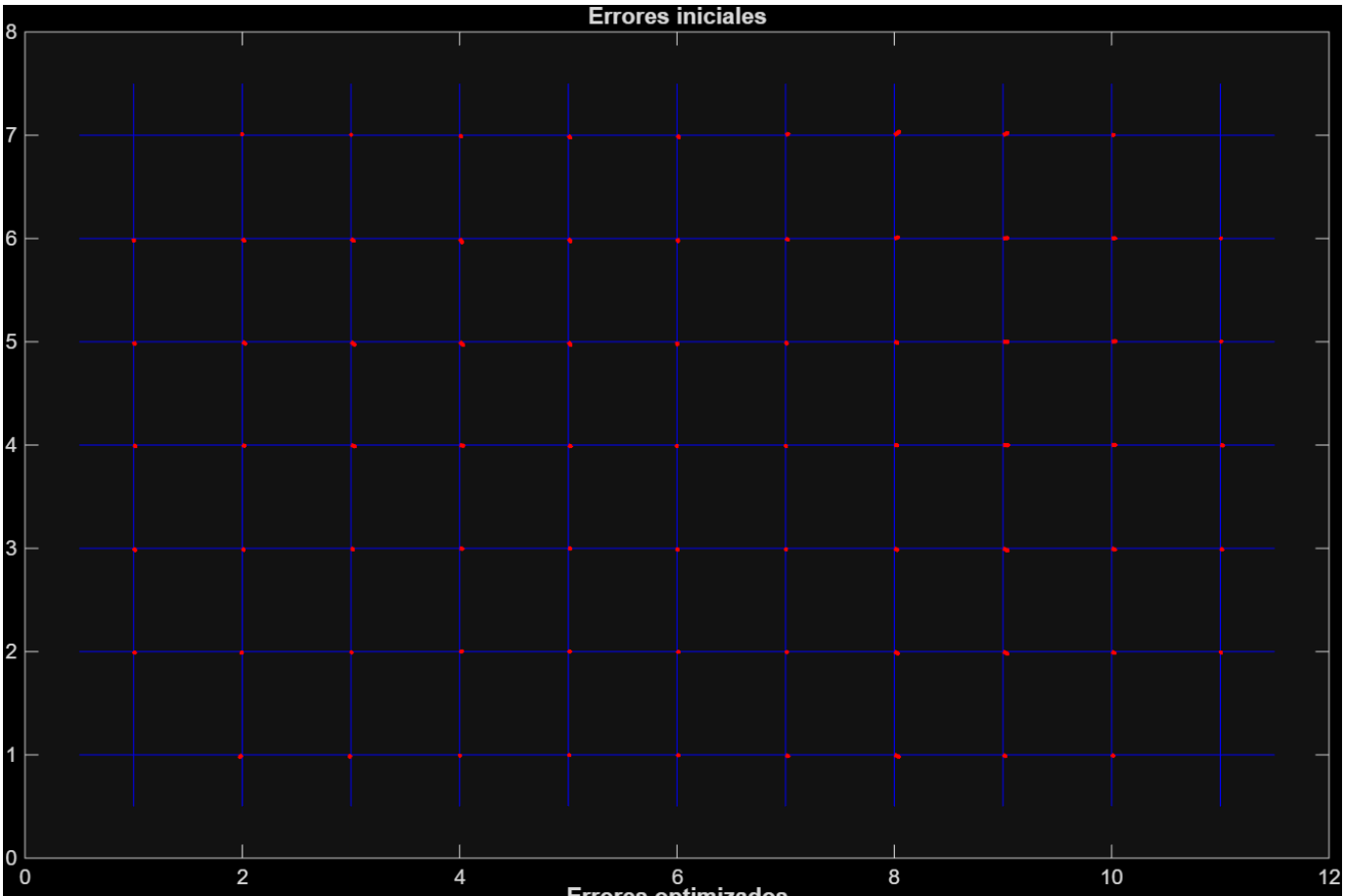
- Valor de f estimado (en pix y mm) a partir de la matriz H.

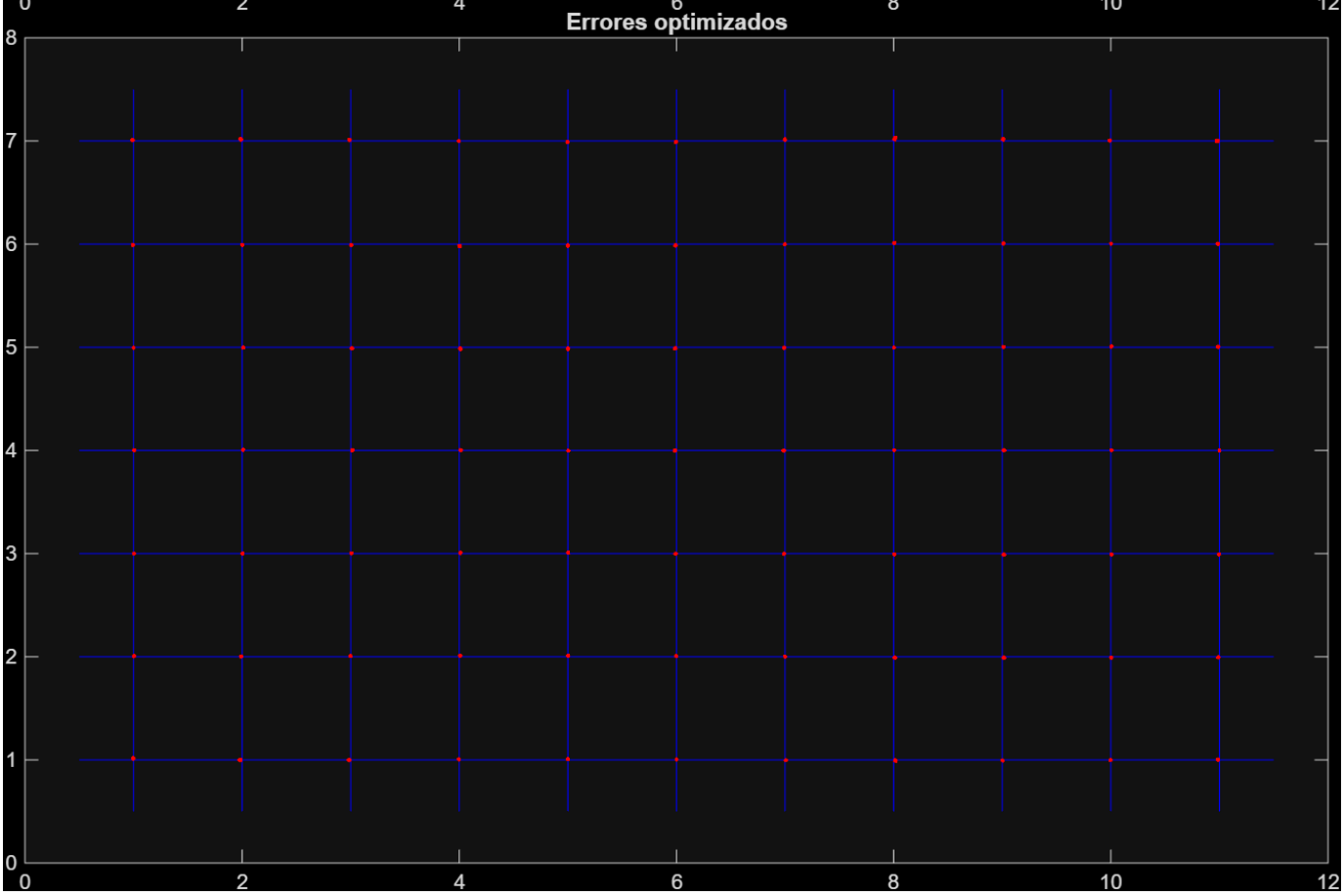
```
f = 1440.48 px
fmm = 6.685 mm
```

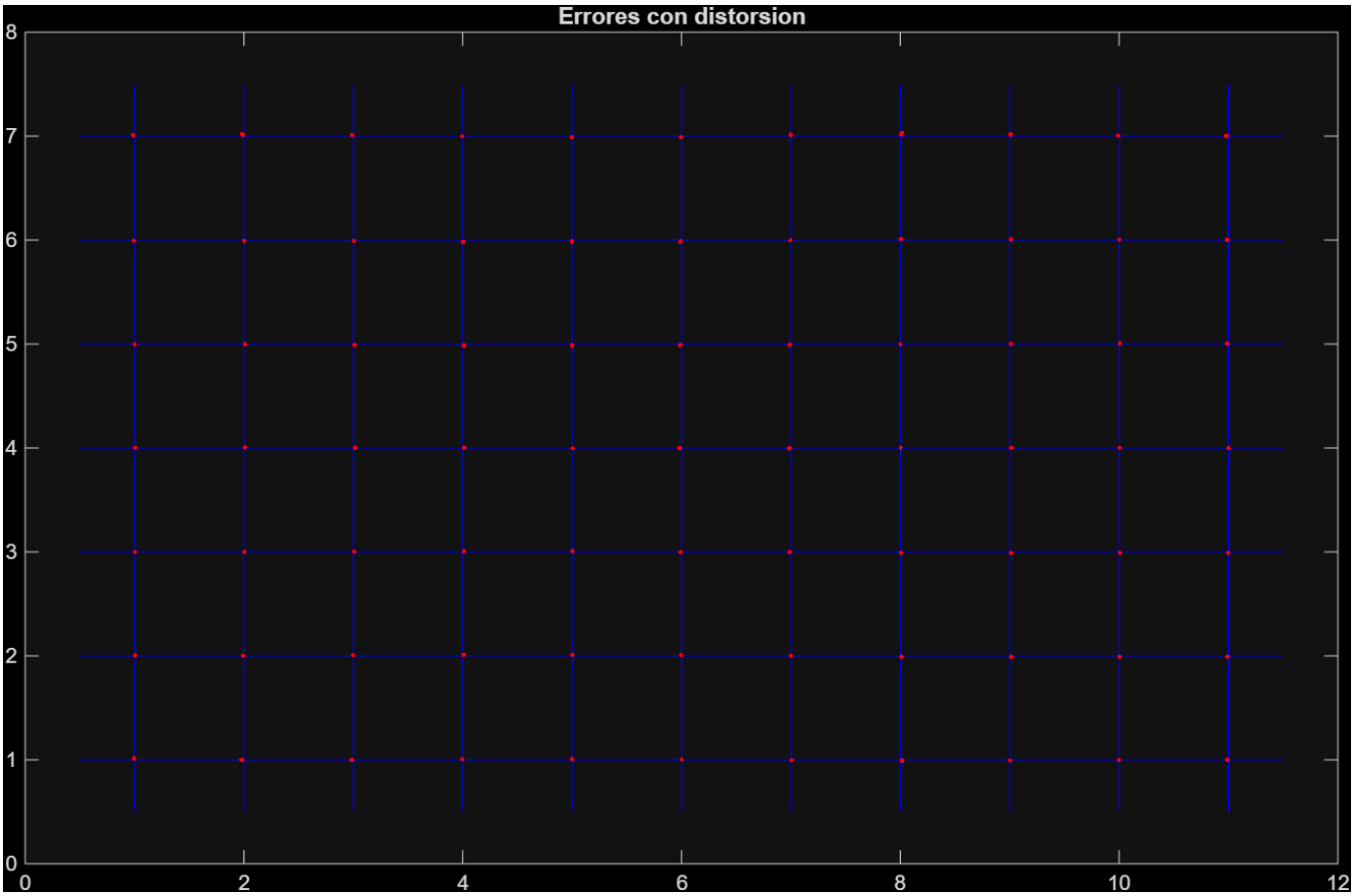
- Resultado de la optimización usando f o (f+k1) si lo habéis implementado.

Focal optimizada: 1304.377 px
k1 = -0.003333

- Gráficos usando show_err_malla() de los errores en las coordenadas (u,v) antes y después de la optimización.


Errores iniciales
0
1
2
3
4
5
6
7
8
0
2
4
6
8
10
12


Errores optimizados
0
1
2
3
4
5
6
7
8
0
2
4
6
8
10
12



- Valores de R y X0 obtenidos en la optimización. Determinar la distancia de la cámara al centro de la malla en el momento de la hacer foto.

```
Matriz R :
    0.9985    0.0058    0.0158
    0.0084   -0.9868   -0.1697
    0.0146    0.1701   -0.9854
Distancia camara-centro malla: 292.62 mm

XO:
-26.055732448182145
-47.787818089839348
2.875126151521980e+02

(Se me olvido imprimirlo para la prueba así que lo he extraído de las variables que
guarda MatLab para no modificar el resto de los números)
```